
Reverse Engineering a Knowledge Worker

A Reference Architecture for Autonomous Knowledge Work

Project REKNOW Technical Report

Suwesh Prasad Sah

Independent engineering work

Project REKNOW

suwesh.github.io

August 2026

Abstract

Structured knowledge work is often performed not as a single act of reasoning but as a stateful sequence of activities that span communication, documentation, task interpretation, planning, interaction with software, evidence collection, and reporting.

This paper presents Project REKNOW, a reference architecture for decomposing structured knowledge work into modular, cooperating software components. The architecture combines work intake, knowledge retrieval, requirement analysis, planning, state-oriented orchestration, computer-using execution, evidence generation, and human communication.

Project REKNOW does not propose a new machine-learning algorithm and does not claim to reproduce human intelligence. Instead, it documents a systems-engineering approach for composing existing technologies into an auditable, extensible, and stateful workflow architecture. An autonomous graphical user-interface quality-assurance workflow is used as the initial reference implementation.

Keywords: Autonomous Knowledge Work, Agentic Systems, Distributed Systems, Workflow Orchestration, Computer-Using Agents, Human-in-the-Loop Systems, Heterogeneous Computing, GUI Automation

Implementation Artifacts: Source code, prompt templates, workflow configurations, and supporting evidence for the reference implementation are available [here](#).

Preprint notice: This manuscript documents independent engineering work and a generalized reference architecture derived from a proof-of-concept implementation. It has not been peer reviewed.

Contents

1	Introduction	4
1.1	Scope	4
1.2	Reference Implementation	4
2	Motivation	4

3	Problem Statement	5
4	Modeling Structured Knowledge Work	6
5	Project REKNOW Reference Architecture	6
5.1	Architectural Overview	6
5.2	Compute Placement	7
5.3	Data and Control Flow	7
6	Artifact-Centric Workflow and Orchestration	7
6.1	Work Artifact Lifecycle	7
6.2	State Model	8
6.3	Cross-Environment Coordination	9
6.4	Failure and Human-Intervention Paths	9
7	Component Design	9
7.1	Work Intake and Communication	10
7.2	Domain Knowledge Retrieval	10
7.3	Requirement Analysis	11
7.4	Workflow Planning	11
7.5	Execution Subsystem	11
7.6	Evidence and Status Management	12
8	Reference Implementation: Autonomous GUI-Based QA	13
8.1	Implementation Overview	13
8.2	Distributed and Heterogeneous Compute Environment	13
8.3	Business Intelligence Pipeline	15
8.4	State and Artifact Bridge	15
8.5	Execution Intelligence	16
8.5.1	Screen Perception	16
8.5.2	Semantic Intent Selection	17
8.5.3	Deterministic Target Resolution	17
8.5.4	GUI Actuation	18
8.5.5	Observation and Validation	18
8.6	Evidence Capture	18
8.7	Implementation Maturity	19
9	Prototype Evaluation and Engineering Trade-offs	20

9.1	Evaluation Scope	20
9.2	Latency and Resource Constraints	20
9.3	Architectural Decisions and Trade-offs	20
10	Limitations, Lessons Learned, and Future Work	21
10.1	Operational Knowledge Coverage	21
10.2	Extended Workflow Execution	21
10.3	Recovery and Human Intervention	21
10.4	Reusable Agent Skills	21
10.5	Evidence Packaging and Final Reporting	22
10.6	Security and Operational Governance	22
10.7	Performance and Deployment	22
11	Conclusion	22
A	Artifact Schemas	24
A.1	Run Metadata Artifact	24
A.2	Requirement Analysis Artifact	24
A.3	Retrieved Context Artifact	25
A.4	Planning Artifact	25
B	State Transition Definitions	25
C	Agent Interface Contracts	25
C.1	Requirement Analysis Agent	26
C.2	Workflow Planning Agent	26
C.3	Communication Agent	26
C.4	Semantic Intent Selector	26
C.5	Atomic Execution Critic	27
C.6	Business Step Judge	27

1 Introduction

Many enterprise workflows require a knowledge worker to receive an unstructured request, consult multiple sources of documentation, reconstruct the relevant operational context, create an execution plan, interact with one or more software systems, observe the resulting state, collect evidence, and communicate the outcome to another person.

Project REKNOW began with a narrower engineering problem: repetitive graphical user-interface testing consumed engineering capacity and required a human tester to repeatedly perform this sequence. The resulting systems question was broader:

Can the observable workflow of a structured knowledge worker be decomposed into reusable, cooperating software components?

Project REKNOW explores this question through a modular reference architecture. The architecture does not treat the knowledge worker as a single general-purpose agent. Instead, it separates knowledge access, requirement analysis, planning, orchestration, execution, observation, evidence generation, and communication into independently replaceable components.

1.1 Scope

This paper presents an engineering reference architecture rather than a new machine-learning algorithm. The contribution is the architectural composition of existing technologies and the definition of boundaries between probabilistic reasoning, deterministic control, durable artifacts, execution environments, and human oversight.

1.2 Reference Implementation

The initial proof of concept applied the architecture to autonomous GUI-based quality assurance. It processed a human work request and supporting documents, reconstructed relevant application knowledge, produced a business-level execution plan, and used a computer-using execution loop to perform a short sequence of mouse and keyboard actions. The proof of concept serves as an implementation example, not as the only intended application of the architecture.

2 Motivation

Project REKNOW originated from a temporary software testing assignment rather than an attempt to construct a general autonomous agent. To learn a complex enterprise application, I was assigned small system-integration and sanity testing tasks for newly implemented changes. Although this work provided useful exposure to the application’s workflows and dependencies, repeatedly performing similar GUI procedures consumed time that could otherwise be applied to my development and systems-engineering assignments.

While carrying out these tests, I was already using a domain-knowledge RAG chatbot that I had developed using the same enterprise application’s user manuals, FAQs and training video transcripts. I used this chatbot to retrieve business-flow knowledge that I did not yet possess and then manually applied that knowledge while testing the application. This created an unusual division of work: the chatbot helped me understand “how to do” what needed to be done, but I

still had to translate that knowledge into a plan, operate the application, collect evidence, and communicate the result.

I initially viewed the remaining problem as GUI automation. However, mouse and keyboard actions with screenshot capture for each atomic GUI action represented only one stage of a larger workflow involving work intake, knowledge retrieval, requirement analysis, planning, execution, observation, evidence collection, and reporting. As I gained experience with workflow automation and application programming interfaces, I began connecting these stages into a stateful system.

This progression produced the first Project REKNOW prototype. Autonomous GUI-based quality assurance became the initial use case through which the broader architecture for structured knowledge work was explored.

3 Problem Statement

Enterprise knowledge systems, workflow automation, and computer-using agents address different parts of structured knowledge work but none of these alone can represent the complete work lifecycle. A knowledge system may retrieve relevant information but cannot turn that information into an executable plan and actions. A workflow automation may coordinate predefined processes but cannot necessarily interpret a new request without explicitly designing workflows for these new requirements. A computer-using agent may interact with software, but it still requires a grounded objective, execution state, domain knowledge, validation, and a mechanism for communicating with humans.

This creates a knowledge-to-action gap. In the reference implementation, new requirements arrive through an unstructured work request including supporting business and functional documents. These artifacts described the intended new changes but did not necessarily contain the established business procedures required to reach the affected stage of the application. Conversely, the existing domain-knowledge base documented baseline workflows but did not contain instructions for functionality that had not yet been introduced.

Closing this gap required more than retrieving documents. The system had to identify the baseline flows related to the requested change, formulate targeted retrieval queries, acquire operational context from the existing knowledge base, and combine that context with the new requirements. The resulting information then had to be transformed into an end-to-end business-level execution plan that preserved both existing process requirements and the behaviour introduced by the change.

A business-level plan was still not directly executable. Each goal had to be progressively translated into atomic software interactions based on the current interface state. The execution subsystem had to observe the application, select one atomic action at a time, resolve semantic targets to physical interface elements, perform mouse or keyboard operations, and preserve screenshots as part of the work product. Evidence generation was therefore not only an observability mechanism for the automation system. It reproduced an expected output of the original (human-based) quality-assurance process, in which execution evidence supports review and sign-off.

The engineering problem addressed by Project REKNOW is therefore the composition of these capabilities into a bounded autonomous workflow. The architecture must coordinate modular components across distributed compute environments, preserve work through durable artifacts and explicit state transitions, and separate probabilistic reasoning from deterministic control. It must also provide traceability, observability, evidence preservation, failure isolation, controlled

access to enterprise resources, and defined points for human intervention.

The objective is not to reproduce the general intelligence, judgment, or organizational role of a human worker. It is to model a bounded, externally observable work process that can operate within defined knowledge, application, permission, and escalation boundaries.

Production viability, however, requires more than technical feasibility: infrastructure cost, documentation quality, and maintenance effort must remain proportionate to the value and volume of the work being automated.

4 Modeling Structured Knowledge Work

Project REKNOW models structured knowledge work by decomposing it into eight functional stages:

1. Receive and register a work request.
2. Acquire relevant operational context.
3. Interpret the requirements.
4. Plan the work.
5. Execute the plan through software systems.
6. Observe the resulting state during execution.
7. Preserve evidence and execution status.
8. Communicate the outcome or request human intervention.

This decomposition was derived from the original GUI-based quality-assurance workflow. It includes cognitive functions such as context acquisition, requirement interpretation, and planning, together with operational functions such as interaction, observation, evidence capture, and communication.

Project REKNOW reverse-engineers these functions at the workflow level. It does not attempt to reproduce the biological or psychological mechanisms through which a human performs them. Each function is instead represented as a bounded system responsibility that can be assigned to an AI model, deterministic software component, human participant, or a combination of these mechanisms.

5 Project REKNOW Reference Architecture

5.1 Architectural Overview

Project REKNOW separates structured knowledge work into collaborating components with explicit responsibilities and durable interfaces. The reference architecture includes work intake, domain knowledge, requirement analysis, planning, state and artifact coordination, software execution, evidence generation, and human communication. Figure 1 illustrates the generalized reference architecture.

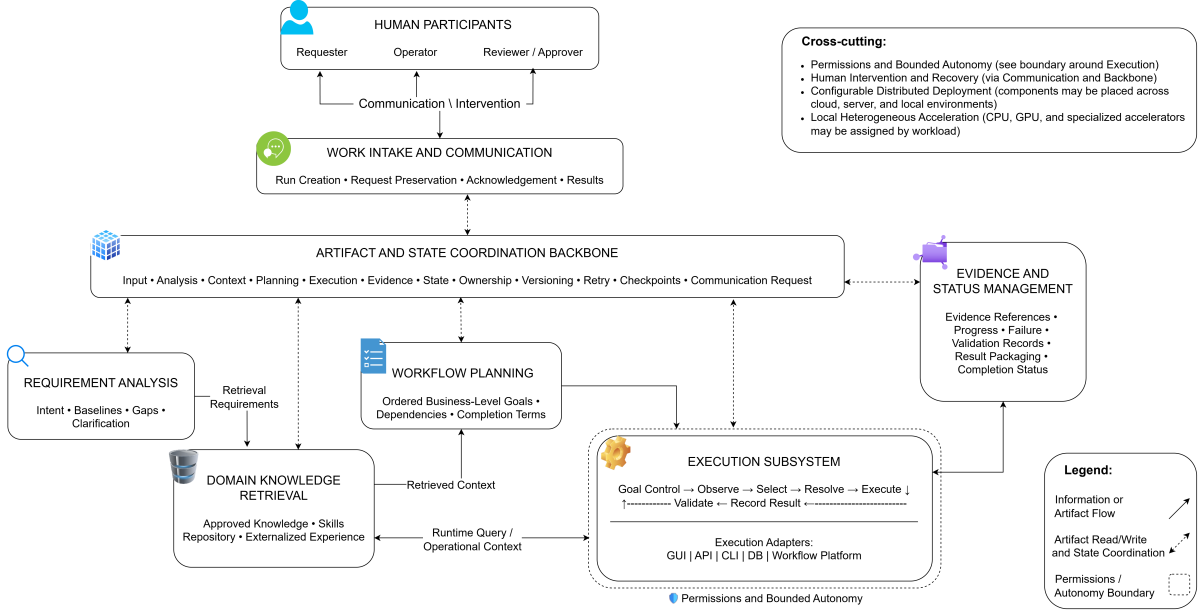


Figure 1: Generalized Project REKNOW reference architecture. Durable artifacts and explicit state coordinate functional components across configurable distributed environments. Individual deployments may use heterogeneous processors or accelerators according to workload requirements.

5.2 Compute Placement

Components may be distributed according to data locality, software access, security constraints, latency requirements, and resource availability. Individual environments may additionally use heterogeneous processors or accelerators according to workload needs.

5.3 Data and Control Flow

Work requests progress through analysis, knowledge retrieval, planning, execution, evidence generation, and communication. Durable artifacts and explicit state transitions form the coordination backbone shared by these components.

6 Artifact-Centric Workflow and Orchestration

Project REKNOW uses an artifact-centric workflow approach rather than relying on a continuous conversation between agents. Each stage receives durable inputs, performs a bounded responsibility, and writes outputs that can be inspected before the next stage begins. A persistent state record coordinates these asynchronous stages across different compute environments.

6.1 Work Artifact Lifecycle

A work request is represented as a run-specific collection of artifacts. The lifecycle begins when the system assigns a unique run identifier and preserves the original request, its attachments, and associated metadata. Subsequent components add new artifacts without replacing the original inputs. The generalized artifact lifecycle contains seven logical artifact classes:

1. **Metadata artifacts** identify the run, requester, communication context, creation time, and source attachments.
2. **Input artifacts** preserve the original work request and supporting documents in forms suitable for downstream processing.
3. **Analysis artifacts** record the interpreted intent, applicable baseline workflows, identified knowledge gaps, and retrieval requirements.
4. **Planning artifacts** describe the ordered business-level goals produced from the requirements, analysis results, and acquired operational context.
5. **Execution artifacts** contain executable instructions, action records, observations, and intermediate execution results.
6. **Evidence artifacts** preserve the outputs required to demonstrate what work was performed and what outcomes were observed.
7. **Status artifacts** represent the current lifecycle state, ownership, timestamps, and transition information for the work run.

This structure creates an intermediate representation of the work at each stage. For example, an analysis artifact is not an executable test plan. It identifies which existing workflows must be understood and which knowledge should be retrieved. The planning component consumes that result and produces a business-level plan. The execution subsystem then translates each business-level goal into operations appropriate to the target software environment. A failed downstream component does not erase the original request or the outputs of completed stages. Artifacts can be inspected manually, reprocessed by compatible components, or used as checkpoints when resuming a workflow from a known state.

These classes define logical responsibilities rather than a mandatory storage structure. An implementation may represent them as files, database records, messages, object-storage entries, or combinations of these mechanisms. In the proof of concept, the classes were instantiated as run-specific folders for metadata, input, analysis, planning, execution, evidence, and status.

6.2 State Model

The artifact lifecycle is coordinated through an explicit run state. The state identifies which stage has completed and which component is permitted to process the run next. A minimal state record contains a run identifier, current lifecycle state, timestamp, and logical owner of the work.

The proof-of-concept implementation used the principal states **RECEIVED**, **ANALYZED**, and **PLANNED**. After the plan was produced, the GUI execution subsystem was invoked through a manual hand-off and demonstrated a sequence of atomic software interactions. The intended lifecycle would introduce explicit **EXECUTING** and **EXECUTED** states, followed by evidence compilation, reporting, and completion. Each state indicated that the corresponding artifacts were available for the next component.

In the generalized architecture, a transition should occur only when the current stage has produced a valid output. The transition therefore acts as both a coordination signal and an artifact-availability contract. For example, the transition to **ANALYZED** implies that an analysis artifact exists and conforms to the interface expected by the retrieval and planning stages.

A production-capable state record should additionally contain a state version, last-update time, current and next owner, retry count, and an optional failure or intervention reason. Versioned state updates are needed to prevent two independently executing components from processing the same transition concurrently or overwriting a newer state.

The state model is intentionally separate from agent reasoning. A language model may produce an analysis or plan, but deterministic orchestration controls whether the output is accepted, persisted, and made available to the next stage.

6.3 Cross-Environment Coordination

Project REKNOW allows components to execute in different environments according to their data/software access, hardware requirements and operational constraints. The reference implementation used three principal compute locations: a cloud environment for work intake and business intelligence, a server environment for orchestration and knowledge retrieval, and a desktop environment for interaction with the target graphical application. The desktop environment additionally used heterogeneous CPU- and GPU-based processing for perception, multimodal inference, and GUI control.

This placement illustrates a general architectural principle: REKNOW components should be located where the required data, hardware, and software interfaces are available. Coordination should occur through durable artifacts and explicit state transitions.

6.4 Failure and Human-Intervention Paths

A stateful workflow must distinguish between incomplete work, failed processing, and work that requires human input. These conditions should not be represented by silently generating an output or advancing to the next lifecycle stage.

The generalized state model therefore includes paths for retry, clarification, pause, escalation, and terminal failure. A transient service error may move a run to a retryable state. Missing or conflicting requirements may require clarification from the requester. An ambiguous or unsupported execution state may pause the run for human intervention. A non-recoverable failure should preserve the completed artifacts and record the stage and reason at which processing stopped.

Human intervention is treated as part of the workflow rather than as an exception outside of the architecture. A communication component may send clarification or escalation messages through the original work-request channel. When a response is received, the run can resume from the relevant state without discarding the preceding artifacts.

7 Component Design

Project REKNOW decomposes structured knowledge work into components with bounded responsibilities and explicit interfaces. Components exchange durable artifacts rather than relying on shared conversational memory. This allows reasoning models, deterministic services, workflow engines, and human participants to be replaced or extended without redesigning the complete system.

The following components define logical responsibilities and do not require a one-to-one mapping to independently deployed services.

7.1 Work Intake and Communication

The Work Intake and Communication component provides an interface between human participants and the autonomous system. Inputs may include natural-language requests, structured forms, attachments, conversation history, requester metadata, and external work-item identifiers. It receives requests, creates a uniquely identified work run, preserves the original communication context, and returns messages through the originating or configured communication channel. Its principal outputs are:

- a unique run identifier;
- communication and requester metadata;
- the original request and supporting attachments;
- acknowledgment or clarification messages; and
- completion, failure, or escalation reports.

Communication may occur at any stage of the workflow, and is not limited to the beginning and end of a run. When another component requires clarification or human intervention, this component delivers the request, correlates the response with the existing run, and stores the response as a new input artifact. Requirement interpretation and execution decisions remain outside its responsibility.

7.2 Domain Knowledge Retrieval

The Domain Knowledge Retrieval component acquires operational context that is not fully contained in the work request. A domain knowledge base should represent, as far as practical, the knowledge available to an experienced human performing the same work. Knowledge sources may include:

- manuals, guidelines and policies;
- frequently asked questions and training material;
- documented business processes and previous work artifacts;
- error-resolution procedures and recovery instructions;
- application-specific shortcuts and alternative procedures; and
- externalized experience from practitioners who perform the knowledge work.

The objective is not to copy a person’s undocumented knowledge automatically. Tacit knowledge, such as workarounds, diagnostic techniques, and application-specific practices, must first be made explicit, reviewed, and maintained as an approved knowledge resource.

The component receives retrieval requirements and returns relevant, bounded context for downstream requirement analysis, planning or intermediate execution decisions. An execution subsystem may also issue additional queries when the current software state exposes an error, an unfamiliar condition, or an unresolved operational choice. Retrieval may use semantic search, keyword search, metadata filtering, relationship expansion, reranking, or a combination of these methods, while preserving the relationship between each returned context and the originating query.

7.3 Requirement Analysis

The Requirement Analysis component determines what must be understood before a work plan can be created. It consumes the original request, supporting documents, metadata, and any initially available operational context. The component identifies:

- the requested outcome;
- applicable baseline workflows;
- new or modified behaviour;
- missing or conflicting information; and
- additional operational context required for workflow planning.

Its output is a structured analysis artifact containing an intent summary, affected areas, applicable workflows, knowledge gaps, and retrieval requirements. The component identifies what must be understood but does not generate the execution plan.

If the request cannot be interpreted without an unsupported assumption, the analysis artifact records a clarification requirement. The clarification is sent to the appropriate human participant through the Work Intake and Communication component. The response is added to the existing run as a new input artifact, together with the analysis already completed. Requirement analysis then resumes using both the preserved analysis context and the newly supplied information.

7.4 Workflow Planning

The Workflow Planning component transforms interpreted requirements and acquired operational context into an ordered set of business-level goals. It consumes the original request, the analysis artifact, the context returned by domain-knowledge retrieval, and any additional human input obtained during clarification.

The planner connects change-specific requirements with established processes required to reach and complete the affected stage of work. Its output describes what must be achieved and in what order without prescribing a particular execution mechanism.

The output of this component, a planning artifact, may contain business-level goals such as reaching a defined process stage, providing required information, invoking a business operation, exercising a new or modified behavior, or verifying an expected outcome. It should not contain screen-element identifiers, API syntax, or other execution-specific details. Resolving these details belongs to the Execution Subsystem.

If required context remains unavailable, the planner records the unresolved dependency instead of generating unsupported steps. The workflow may then issue an additional knowledge-retrieval request or obtain clarification from a human participant through the Work Intake and Communication component. Planning resumes after the missing context has been added to the work run.

7.5 Execution Subsystem

The Execution Subsystem converts business-level goals into operations supported by the target software environment. It consumes a planning artifact, the current execution state, available

software interfaces, additionally retrieved operational context, and the permissions assigned to the work run. A business-level goal may require a sequence of smaller operations. Execution therefore proceeds iteratively:

1. inspect the current software state;
2. select an operation that progresses the current goal or step;
3. resolve the operation against an available interface;
4. perform the operation;
5. observe and record the resulting state;
6. evaluate progress toward the current goal; and
7. determine whether to continue, retry, pause, or request human intervention.

This loop represents atomic progression without requiring every implementation to define an atomic operation in the same way. An atomic operation may be a GUI action, API request, workflow invocation, database transaction, command-line operation, or another indivisible interaction supported by the execution environment. Where practical, semantic decisions should remain separate from deterministic execution. A reasoning model may select the intended operation, while deterministic software validates its structure, resolves the target, enforces applicable constraints, and performs the action.

The Execution Subsystem may maintain an internal state machine for the active business goal. This local state tracks the current goal, completed operations, observations, retries, and goal-completion status. It remains subordinate to the run-level state model: the local execution state tracks progress within the active business goal, while the run-level state tracks progress across the complete REKNOW lifecycle.

The subsystem produces action or operation records, observations, intermediate results, goal-completion status, and references to captured evidence. An unfamiliar condition may trigger an additional domain-knowledge query. An ambiguous or unsupported state produces an explicit pause, failure, or intervention request rather than an unrestricted continuation attempt.

7.6 Evidence and Status Management

The Evidence and Status Management component maintains the observable record of a work run and exposes its current progress to monitoring, communication, and review mechanisms.

Evidence records what was performed or observed. Depending on the work domain, evidence may include screenshots, generated documents, software responses, execution traces, timestamps, and validation records. Each artifact is associated with the relevant run, business goal, and execution operation. Evidence capture does not itself establish correctness. An evidence artifact preserves an observation, while a validation mechanism determines whether that observation satisfies the corresponding goal.

Status records whether the run is progressing, waiting, paused, failed, or complete. It also preserves failure reasons, intervention requests, and completion information required by the communication component. Incomplete, paused, and failed outcomes therefore remain explicit lifecycle states and cannot be reported as successful completion.

8 Reference Implementation: Autonomous GUI-Based QA

The reference implementation applied Project REKNOW to autonomous graphical user-interface quality assurance for a custom enterprise web application. The prototype processed a human test request and supporting requirement documents, reconstructed the existing business flows needed to exercise the requested change, produced an end-to-end business-level test plan, and executed part of that plan through visually grounded mouse and keyboard interactions.

The implementation is described using generalized organizational and application terminology (see Figure 2). Email addresses, internal application names, business identifiers, infrastructure addresses, and organization-specific workflow details have been removed.

8.1 Implementation Overview

The prototype instantiated the generalized architecture through three cooperating subsystems:

1. a cloud-based business-intelligence subsystem for work intake, requirement analysis, planning, and human communication;
2. a server-side state and artifact bridge responsible for document extraction, domain-knowledge retrieval, state observation, and cross-environment coordination;
3. and a desktop-based execution-intelligence subsystem responsible for observing and operating the target enterprise application.

A human requester initiated a run by sending an email containing test instructions and supporting business and functional requirement documents. The system created a run-specific artifact workspace, preserved the request and attachments, and initialized a status artifact. The persisted run metadata followed the schema defined in Appendix A.1. Specialized language-model components analyzed the request, generated domain-knowledge queries, and produced an ordered set of business-level test goals.

The server-side subsystem connected the cloud workflow to the existing domain knowledge base. It observed state changes, extracted source documents, executed retrieval requests, and wrote the resulting context back into the shared workspace. After planning completed, the generated plan was passed to the desktop execution subsystem.

The server-to-desktop handoff remained manual in the demonstrated prototype. Once invoked, however, the desktop subsystem implemented the complete perception, intent-selection, target-resolution, actuation, and screenshot-capture loop required to progress a business-level goal through atomic GUI actions.

Once the plan was successfully generated, the system also sent an acknowledgment in the original email thread. The acknowledgement summarized and attached the generated test plan artifact to the response.

8.2 Distributed and Heterogeneous Compute Environment

The implementation distributed processing across three compute environments according to software access, data locality, hardware requirements, and operating-system constraints.

The first environment used cloud-hosted services Power Automate workflows and AI Builder prompts. This environment had access to the Outlook email channel and shared SharePoint ar-

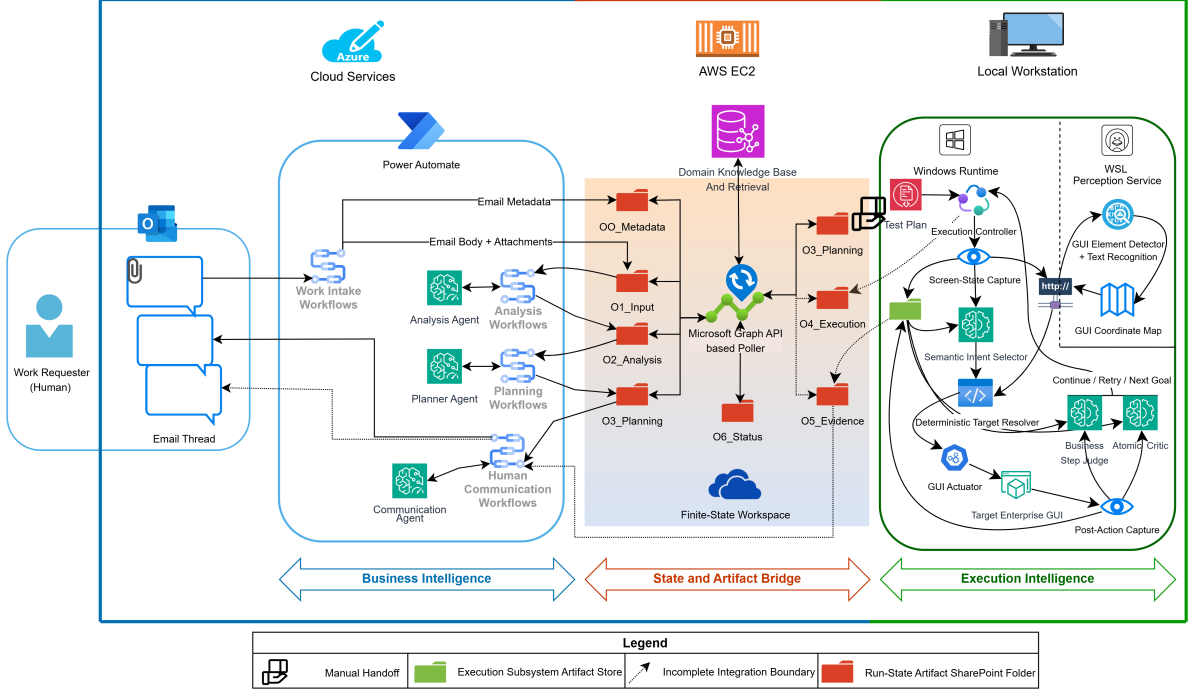


Figure 2: Project REKNOW reference implementation for autonomous GUI-based quality assurance. The prototype distributes business intelligence, state and artifact coordination, and execution intelligence across cloud services, an AWS EC2 environment, and a local Windows/WSL workstation.

tifact workspace. It hosted work intake, workspace initialization, requirement analysis, workflow planning, and acknowledgment generation.

The second environment used a GPU-enabled AWS EC2 server. It hosted the Microsoft Graph API poller, document-extraction functions, run-state handling, FAISS-based domain retrieval (embedding and reranking), and the existing enterprise knowledge base. Locating the poller and retrieval pipeline in the same server environment reduced the number of interfaces required between state coordination and knowledge access. The EC2 instance provided a continuously available server environment for the poller and retrieval services during prototype operation.

The third environment was a local workstation containing an NVIDIA RTX 3060 GPU with 12 GB of VRAM. Windows provided the authenticated browser session, full-screen screenshot capture, and mouse and keyboard control. LM Studio hosted the Qwen3-VL-8B multimodal model used for semantic action selection. A WSL environment on the same workstation hosted the GPU-assisted screen-perception service. This service used the YOLO-based Salesforce GPA-GUI-Detector and TrOCR to transform screenshots into structured screen maps. Windows submitted screenshots to this service through a local HTTP interface and received detected interface elements and their coordinates in return. The operating-system split was therefore functional rather than decorative. The Windows environment provided access to the enterprise GUI and desktop-control interfaces, while the Linux environment provided a practical runtime for the visual detection and OCR models. The local HTTP boundary allowed both environments to remain independently executable while sharing the same physical GPU workstation.

8.3 Business Intelligence Pipeline

The business-intelligence pipeline transformed an incoming request into a business-level test plan through a sequence of durable artifacts.

When a new email with attachments arrived, the initialization workflow:

1. sanitized the email subject for use in artifact paths;
2. generated a unique run identifier;
3. created the run-specific artifact folders;
4. preserved email metadata and plain-text message content;
5. stored the supporting attachments; and
6. initialized the run-state artifact.

The server-side handler for the received state downloaded the email and supported PDF and DOCX attachments. Text was extracted and combined into a single analysis-input artifact. Creating this normalized artifact triggered the Requirement Analysis Agent to process the request without depending on the source document format.

The Requirement Analysis Agent was implemented as a specialized AI Builder prompt. Its responsibility was limited to contextual discovery, identifying the request’s intent, relevant existing baseline workflows, and the required knowledge before an execution could be planned. Appendix A.2 defines the corresponding JSON contract.

For each generated query, the server executed retrieval over three knowledge collections: frequently asked questions, training-video transcripts, and application user manuals. The collections were searched in parallel using query embeddings. Retrieved items were combined and reranked as a common candidate set. The highest-ranked manual chunks were then expanded using their relationship to neighboring manual units so that the planner received coherent procedural context rather than isolated fragments. The Retrieved Context Artifact contract is defined in Appendix A.3.

The Workflow Planning Agent received the original request and requirement documents, the structured Analyst output, and the retrieved domain context. It produced an ordered JSON planning artifact (Appendix A.4) whose entries contained a step identifier and a business-level goal. The planning artifact contained goals covering the existing end-to-end process as well as the newly introduced functionality. The goals remained independent of interface controls and coordinates. For example, the plan could require the system to create a new application, complete onboarding prerequisites, reach a defined business stage, exercise the changed behavior, and verify downstream outcomes. The Execution Intelligence subsystem was responsible for decomposing each such goal into atomic GUI operations.

After the planning artifact was created, the Communication Agent generated a human-readable acknowledgment. The response summarized the planned test coverage and attached the planning artifact, confirmed that testing would proceed, and was sent as a reply in the original email thread.

8.4 State and Artifact Bridge

The cloud and server environments shared a run-specific artifact workspace implemented using SharePoint and accessed through Microsoft Graph API. The server-side bridge maintained two

related records:

- the run-state artifact folder stored with each run; and
- a processed-state tracker recording which state values had already been handled for each run.

The poller enumerated available run folders, located the corresponding status artifacts, and compared each observed state against the processed-state tracker. A state that had already been handled for the same run was skipped. A newly observed state was dispatched to the corresponding deterministic handler and was recorded in the tracker after processing.

The principal handler sequence was:

1. **RECEIVED**: extract and normalize the email and supporting documents, then write the analysis-input artifact;
2. **ANALYZED**: read the Analyst output, execute the generated knowledge queries, and write the retrieved context for planning; and
3. **PLANNED**: read and validate the generated planning artifact in preparation for execution.

All components reacted to the creation of stage-specific trigger artifacts, invoked their respective processes, persisted their structured outputs, and updated the run state. The poller then observed the new state and prepared the next artifact.

After the **PLANNED** handler successfully generated the execution plan and the acknowledgment was autonomously communicated to the requester, the final server-to-desktop invocation however was not wired into the demonstrated run. The plan was transferred manually to the desktop execution environment. This was an incomplete integration boundary rather than a missing execution subsystem: the same state-dispatch mechanism could be extended to invoke an execution adapter, but that additional transition was outside the demonstrated PoC.

8.5 Execution Intelligence

The execution subsystem operated one business-level goal at a time. A single goal required multiple iterations of perception, semantic intent selection, deterministic grounding, physical actuation, and observation.

8.5.1 Screen Perception

At the beginning of each iteration, the Windows runtime captured the current desktop image. The screenshot was sent to a FastAPI service running under WSL.

The perception service used the Salesforce/GPA-GUI-Detector to identify candidate interface regions. For the demonstrated configuration, inference used a confidence threshold of 0.05, and input image size of 1280 pixels, and a non-maximum-suppression intersection-over-union threshold of 0.7.

Each detected region was cropped from the original screenshot and passed to TrOCR. The resulting screen map represented each region using:

- an element identifier;

- detected visible text, or the generic value `icon`;
- the center coordinate of the region; and
- its bounding box.

The detector did not classify the semantic type of unidentified icons. Elements without readable text were therefore represented generically. The raw screenshot and a machine-annotated detector image were stored separately. The annotated image served as a perception-debug artifact, while the raw screenshots formed part of the execution evidence.

8.5.2 Semantic Intent Selection

Qwen3-VL-8B served as the semantic intent selector. When available, the model received the previous screen, the current screen, the previous action, and the active business-level goal. The model selected one semantic operation rather than predicting screen coordinates. Its output followed a restricted JSON contract:

```
{
  "intent": "click | type | scroll | wait",
  "target_text": "visible interface text",
  "type_value": "value when the intent is type",
  "reason": "short explanation"
}
```

Restricting the output to one operation forced incremental execution and reduced the amount of unchecked behavior represented by any individual model response. The model decided what should be attempted next, while the actor determined where and how the operation should be executed.

8.5.3 Deterministic Target Resolution

The structured screen map was consumed by a deterministic Python actor. The actor normalized the model-selected target text and first searched for an exact match among the detected interface elements. When no unique exact match existed, the actor used fuzzy text similarity to rank candidate elements. Non-actionable regions and generic icons were excluded where appropriate. Remaining ambiguities were resolved using bounding-box geometry and screen-position heuristics.

The final implementation preferred smaller actionable regions over larger containers because broad detected areas could cover search bars, section containers, or multiple nested controls. A geometric tie-breaker then selected among equally ranked candidates. The center of the selected bounding box became the coordinate supplied to the actuation layer.

This separation was introduced because the local 8B multimodal model did not reliably produce precise coordinates, even when detailed screen maps were made available. Semantic target selection was therefore assigned to the model, while coordinate resolution remained deterministic and inspectable.

8.5.4 GUI Actuation

The Windows runtime used PyAutoGUI to perform the resolved operations. The actor supported:

- clicking a resolved interface element;
- focusing an input region and typing a model-selected value;
- scrolling the current view; and
- waiting when no safe progress operation was available.

After each action, the runtime allowed the interface to settle before capturing the next screenshot. The action outcome recorded the semantic target and resolved coordinate so that an execution trace could be correlated with the saved visual state.

8.5.5 Observation and Validation

The execution loop preserved screen states before and after every atomic action. The previous action and the resulting screen state were supplied to the next intent-selection iteration, allowing the model to adapt its next choice to visible progress.

Two validation roles were also implemented and tested independently.

The Atomic Execution Critic compared the pre-action and post-action screenshots and classified the last action as **SUCCESS**, **FAIL**, or **NO_PROGRESS**. The Critic evaluated only the attributable visual result of the most recent atomic operation.

The Business Step Judge evaluated whether the active business-level goal had been completed and whether the interface was ready to begin the next goal. This was intentionally separate from atomic-action criticism: successful completion of one mouse or keyboard operation does not imply completion of the broader business step.

The Critic and Step Judge were individually validated, but continuous invocation was disabled during the recorded multi-action demonstration. On the available RTX 3060 workstation, each additional multimodal validation call materially increased the execution time. Their absence from the full demonstration therefore reflected a runtime configuration choice under hardware constraints rather than the absence of validation components.

8.6 Evidence Capture

The screenshots captured before and after every actuation were stored under a directory associated with the active business-level step. Filenames encoded the iteration number and actor outcome, including the selected target and resolved coordinates where applicable.

This produced a chronological visual record of the execution sequence. The demonstrated run preserved approximately 14–15 continuous atomic interactions covering interface selection, text entry, navigation, and form progression. A .mp4 artifact was then compiled from the ordered screenshots to present the execution sequence as a continuous demonstration.

Automatic task completion communication with the compiled .mp4 artifact attachment to the original email thread was not exercised because the demonstrated run did not reach the end of the generated business-level plan.

8.7 Implementation Maturity

Table 1 summarizes the maturity of the principal components at the end of the proof of concept.

Capability	Status	PoC evidence
Work intake and run initialization	Implemented	Email-triggered workspace creation, meta-data preservation, attachment storage, and state initialization.
Requirement analysis	Implemented	Structures intent, baseline-flow, and retrieval-query artifact.
Domain knowledge retrieval	Implemented	Parallel retrieval from FAQs, training video transcripts, and manuals, followed by cumulative reranking and manual-context expansion.
Business-level planning	Implemented	Structured end-to-end planning artifact containing ordered business goals.
Email acknowledgement	Implemented	Same-thread response containing a generated plan summary, with planning artifact attached.
State and artifact bridge	Implemented	Graph API polling, processed-state tracking, deterministic handler dispatch, and state-triggered artifact creation.
Automated desktop handoff	Partially implemented	The <code>PLANNED</code> handler retrieved the plan; desktop invocation remained a manual boundary.
Screen perception	Implemented	YOLO-based GUI element detection, OCR, structured UI-map generation, and annotated debug artifacts.
Semantic intent selection	Implemented	Qwen3-VL-8B produced incremental click, type, scroll, and wait operations from visual state and business-goal context.
Deterministic GUI actuation	Implemented	Exact and fuzzy target matching, geometric disambiguation, and PyAutoGUI-based mouse and keyboard operations.
Atomic-action criticism	Demonstrated	Critic prompt, multimodal comparison, structured verdict, and independent validation.
Business-step judgment	Demonstrated	Goal-completion gate and structured output validated independently.
Atomic evidence capture	Implemented	Pre-action and post-action screenshots, detector overlays, action-oriented filenames, and a compiled execution video.
Final evidence delivery	Not exercised	Final evidence same-thread delivery depended on fully completing the generated execution plan.

Table 1: Implementation maturity of the autonomous GUI-based quality-assurance reference implementation.

9 Prototype Evaluation and Engineering Trade-offs

9.1 Evaluation Scope

The prototype evaluation covered the integrated workflow from email-based work intake through requirement analysis, domain-knowledge retrieval, business-level planning, acknowledgment generation, and state-based artifact coordination. GUI execution was evaluated after a manual handoff of the generated planning artifact to the desktop runtime.

The execution evaluation focused on the first business-level goal. The prototype completed 15 consecutive atomic GUI operations, including option selection, text entry, navigation, and form submission. Evidence was preserved throughout the sequence.

The evaluation did not cover automatic server-to-workstation invocation, execution of the full test plan, or automatic delivery of a complete evidence package. The Atomic Execution Critic and Business Step Judge were validated independently but were not continuously invoked during the recorded multi-action run.

9.2 Latency and Resource Constraints

The execution runtime used Qwen3-VL-8B on an NVIDIA RTX 3060 with 12 GB of VRAM. During the original proof-of-concept period, a multimodal action-selection request containing the system prompt and screenshots required approximately one minute. The 15-action sequence therefore required approximately 15 minutes with the Atomic Execution Critic and Business Step Judge disabled. Continuous validation would have introduced additional multimodal model calls after individual actions and business-step boundaries. These components were disabled during the continuous demonstration to keep execution time practical on the available hardware.

This measurement represents the original prototype environment rather than an intrinsic performance characteristic of the model. Inference latency depended on the model-serving software, model version, image processing path, and runtime configuration.

9.3 Architectural Decisions and Trade-offs

The prototype was shaped by four principal architectural decisions, each with corresponding benefits and implementation costs.

- **Specialized components instead of a monolithic agent.** Separating analysis, retrieval, planning, communication, and execution produced clearer responsibilities and inspectable outputs, at the cost of additional interfaces and coordination logic.
- **Durable artifacts instead of transient model context.** Persisting intermediate outputs improved traceability and supported processing across different compute environments, while requiring explicit artifact and state management.
- **Semantic intent selection with deterministic coordinate resolution.** Assigning semantic action selection to the multimodal model and coordinate resolution to deterministic software improved actuation consistency, while making execution dependent on GUI detection, OCR, and target-matching quality.
- **Local inference instead of hosted inference.** Local inference retained control over application screenshots and avoided per-request service costs, but constrained model size,

execution latency, and validation frequency. A larger hosted model could provide greater model capacity and potentially stronger visual reasoning and action selection, but would introduce external-service dependency and operating cost. This alternative was not evaluated end to end in the prototype.

10 Limitations, Lessons Learned, and Future Work

10.1 Operational Knowledge Coverage

The available knowledge base primarily documented business-user workflows. Specialist QA knowledge, including diagnostic techniques, recovery procedures, application shortcuts, inspection methods, and known workarounds, was not sufficiently externalized.

This limited the execution subsystem’s ability to respond to situations that an experienced tester might resolve using tacit operational knowledge. Future implementations should capture this knowledge with experienced practitioners and maintain it as reviewed and versioned operational guidance.

10.2 Extended Workflow Execution

The prototype demonstrated 15 consecutive atomic actions but did not complete the full generated business-level plan. Longer workflows increase the probability of incorrect actions, unexpected interface states, expired sessions, application delays, and accumulated execution errors.

Extended execution therefore requires bounded retries, stable checkpoints, goal-level validation, interface-readiness checks, and resumable execution. The independently tested Atomic Execution Critic and Business Step Judge provide a foundation for this control loop, but their continuous use must be balanced against inference latency and available compute resources.

10.3 Recovery and Human Intervention

Clarification, retry, pause, escalation, and resumable execution were represented in the generalized architecture but were not fully connected in the proof of concept.

Future implementations should define explicit recovery states and intervention policies. Recoverable application errors may be resolved using approved operational guidance, while ambiguous requirements, unsupported interface states, or actions outside the permitted autonomy boundary should be routed to a human participant. The resulting response should be added to the existing run so that execution can resume without discarding completed work.

10.4 Reusable Agent Skills

The prototype encoded agent behavior through specialized prompts, structured schemas, deterministic handlers, and domain-knowledge retrieval. These mechanisms were sufficient for the proof of concept but did not package recurring operational procedures as independently versioned capabilities.

Future versions may represent repeatable procedures as reusable agent skills. A skill definition may contain its purpose, prerequisites, instructions, permitted tools, input and output contracts, validation criteria, and recovery guidance. This would allow frequently used procedures and

specialist operational knowledge to be maintained separately from general prompts and retrieved documentation.

10.5 Evidence Packaging and Final Reporting

Atomic screenshot capture and execution-video compilation were implemented. The prototype did not automatically compile the captured artifacts into a formal sign-off package or deliver the completed evidence through the original communication channel.

A complete implementation should associate evidence with the relevant business goal, operation, execution result, and validation outcome. After the run reaches a terminal state, the system should compile the evidence, generate a concise pass, fail, or incomplete summary, and return the result through the communication channel that initiated the work.

10.6 Security and Operational Governance

The prototype used a pre-authenticated desktop session and did not implement production-grade credential lifecycle management. The shared artifact workspace and captured screenshots could also contain sensitive business information.

A production implementation would require managed credentials, least-privilege execution accounts, authenticated communication between compute environments, and access controls for work artifacts. Evidence retention, screenshot redaction, audit logging, and deletion policies would need to follow the requirements of the deployment environment. High-impact or irreversible operations should additionally require an explicit approval boundary before execution.

10.7 Performance and Deployment

The reference implementation distributed processing across cloud, server, and local workstation environments according to application access, knowledge locality, model compatibility, and GPU availability. Within the workstation, Windows and WSL provided separate operating-system contexts for GUI control and GPU-assisted perception. This placement was effective for the proof of concept but introduced multiple service and operating-system boundaries.

Future deployment decisions should consider model-serving latency, GPU memory capacity, validation frequency, artifact-transfer overhead, availability requirements, and operating cost. Local inference offers greater control over application screenshots and model execution, while hosted inference may provide access to larger models and stronger computer-use capabilities. A production deployment may also separate retrieval, orchestration, model serving, and execution into independently scalable services where the workload justifies the additional operational complexity.

11 Conclusion

Project REKNOW presents a reference architecture for decomposing structured knowledge work into cooperating components for work intake, knowledge retrieval, requirement analysis, planning, orchestration, software execution, evidence generation, and human communication. The architecture combines probabilistic reasoning with deterministic control and uses durable artifacts and explicit state transitions to coordinate work across distributed compute environments.

The autonomous GUI-based quality-assurance prototype demonstrated this approach from email-based request intake through analysis, contextual retrieval, business-level planning, acknowledgment generation, and a 15-action sequence of visually grounded software interactions. The prototype also demonstrated the value of separating semantic action selection from deterministic target resolution and actuation.

The implementation did not complete the full generated test plan or the automatic delivery of a final evidence package. It nevertheless showed that structured knowledge work can be represented as a bounded, inspectable, and partially executable software workflow. Its limitations further showed that model capability alone is insufficient: operational knowledge coverage, validation, recovery, security, compute resources, and deployment cost are equally important to production viability.

The principal contribution of Project REKNOW is therefore not a new model or automation technique, but a systems-engineering composition of existing capabilities into a modular knowledge-to-action architecture. The reference implementation provides a concrete foundation from which the architecture can be evaluated, extended, and applied to other forms of structured knowledge work.

A Artifact Schemas

The schemas below document the principal contracts used by the reference implementation. Identifiers and values are illustrative and contain no organization-specific information.

A.1 Run Metadata Artifact

The run metadata artifact preserves the identifiers required to associate the work request, shared artifact workspace, and original communication thread.

```
{
  "run_folder_id": "sanitized-subject__RUN-YYYYMMDD-HHMMSS",
  "subject": "Original work-request subject",
  "conversation_id": "generalized-conversation-id",
  "message_id": "generalized-message-id",
  "from": "requester@example.com",
  "to": "automation@example.com",
  "cc": "",
  "received_time": "ISO-8601 timestamp"
}
```

A.2 Requirement Analysis Artifact

The analysis artifact records the interpreted intent, applicable baseline workflows, and retrieval requirements. The reference implementation produced retrieval requirements as natural-language queries.

```
{
  "intent_summary": "Summary of the requested change and test intent",
  "baseline_flows": [
    {
      "flow_name": "Existing operational workflow",
      "reason": "Why the workflow remains relevant"
    },
    ...
  ],
  "retrieval_queries": [
    {
      "query": "Operational knowledge required before planning"
    },
    ...
  ]
}
```


A.3 Retrieved Context Artifact

The retrieval artifact preserves the relationship between each generated query and the context returned from the available knowledge collections.

```
{
  "rag_context": [
    {
      "query": "Operational knowledge required before planning",
      "faqs_context": "Relevant FAQ content or None",
      "videos_context": "Relevant transcript content or None",
      "manuals_context": "Expanded manual context or None"
    },
    ...
  ]
}
```

A.4 Planning Artifact

The planning artifact represents the ordered business-level goals produced from the request, analysis artifact, and retrieved context.

```
{
  "business_steps": [
    {
      "step_id": 1,
      "goal": "Business-level outcome to be achieved"
    },
    {
      "step_id": 2,
      "goal": "Next business-level outcome"
    },
    ...
  ]
}
```

B State Transition Definitions

Refer to [Table 2](#)

C Agent Interface Contracts

The reasoning components used specialized prompts with restricted responsibilities and structured outputs. The complete sanitized prompt templates and representative implementation code are maintained in the Project REKNOW repository.

State	Definition
RECEIVED	The work request, metadata, and supporting input artifacts have been preserved and are available for analysis.
ANALYZED	Requirement analysis has completed, and the analysis artifact and retrieval requirements are available.
PLANNED	Domain context has been retrieved, and an ordered business-level planning artifact is available for execution.

Table 2: Implemented Run-level state definitions.

C.1 Requirement Analysis Agent

Input: Original work request, supporting requirement documents, and available run context.

Responsibility: Interpret the requested change, identify applicable baseline workflows, and generate retrieval queries for missing operational context.

Restriction: The component does not generate execution steps or select software actions.

Output: Requirement Analysis Artifact (Appendix A.2).

C.2 Workflow Planning Agent

Input: Original request, Requirement Analysis Artifact, retrieved domain context, and any clarification supplied by a human participant.

Responsibility: Connect change-specific requirements with baseline workflows and produce an ordered sequence of business-level goals.

Restriction: The component does not generate GUI coordinates, screen-element identifiers, or atomic software operations.

Output: Planning Artifact (Appendix A.4).

C.3 Communication Agent

Input: Communication context and the current planning, execution, or status artifact.

Responsibility: Generate an acknowledgment, clarification request, escalation message, or result summary for delivery through the configured communication channel.

Restriction: The component does not modify the plan or determine execution status.

C.4 Semantic Intent Selector

Input: Current screenshot, previous screenshot, previous action, and active business-level goal.

Responsibility: Select one semantic operation from `click`, `type`, `scroll`, or `wait`.

Restriction: The component does not generate screen coordinates or execute the selected operation.

C.5 Atomic Execution Critic

Input: Pre-action screenshot, post-action screenshot, and the executed operation.

Responsibility: Classify the observable outcome as `SUCCESS`, `FAIL`, or `NO_PROGRESS`.

Restriction: The component evaluates only the latest atomic operation and does not determine completion of the business-level goal.

C.6 Business Step Judge

Input: Active business-level goal, current screen state, and recent execution context.

Responsibility: Determine whether the active business-level goal has been completed and whether execution may proceed to the next goal.

Restriction: The component does not perform GUI operations or replace atomic-action validation.